

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0714
NAME: KEERTHANA R
REG NO:192524489

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment` (`length_m`) to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length

of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, wellorganized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments

Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done
---------------------	----	--	------------------------	-----------------	-----------------

3) Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

ANSWER:

Python Program

```
import random
```

```
HOSTS = ["Host 1", "Host 2", "Host 3", "Host 4"]
```

```
TOTAL_FRAMES = 12
```

```
MAX_COLLISIONS = 6
```

```
random.seed(10)
```

```
events = []
```

```
queue = []
```

```
for i in range(TOTAL_FRAMES):  
    host = HOSTS[i % len(HOSTS)]  
    queue.append({  
        "host": host,  
        "frame": i + 1,  
        "attempts": 0,  
        "collisions": 0,  
        "ready": random.randint(0, 3)  
    })
```

```
time = 0
successful = 0
collisions = 0
```

```
with open("csma_log.txt", "w") as log:
```

```
    log.write("CSMA/CD SIMULATION LOG\n")
    log.write("=" * 50 + "\n")
```

```
    while queue:
```

```
        ready_hosts = [x for x in queue if x["ready"] <= time]
```

```
        if not ready_hosts:
```

```
            time += 1
```

```
            continue
```

```
        log.write(f"\nTime Slot {time}\n")
```

```
        # All ready hosts sense the channel and attempt transmission
```

```
        for item in ready_hosts:
```

```
            item["attempts"] += 1
```

```
            log.write(
```

```
                f"TRANSMISSION ATTEMPT: {item['host']} "
```

```
                f"Frame {item['frame']} Attempt {item['attempts']}\n"
```

```
            )
```

```
        if len(ready_hosts) == 1:
```

```
            item = ready_hosts[0]
```

```
            log.write(
```

```
                f"SUCCESS: {item['host']} transmitted "
```

```
                f"Frame {item['frame']}\n"
```

```
            )
```

```
successful += 1
queue.remove(item)
```

```
else:
```

```
    collisions += 1
```

```
    log.write(
        f"COLLISION: {len(ready_hosts)} hosts attempted "
        f"transmission simultaneously\n"
    )
```

```
    for item in ready_hosts:
        item["collisions"] += 1
```

```
    if item["collisions"] > MAX_COLLISIONS:
        log.write(
            f"DROP: {item['host']} Frame {item['frame']} "
            f"exceeded maximum collision limit\n"
        )
        queue.remove(item)
        continue
```

```
    k = min(item["collisions"], 5)
    max_backoff = (2 ** k) - 1
    backoff = random.randint(0, max_backoff)
```

```
    item["ready"] = time + backoff + 1
```

```
    log.write(
        f"BACK-OFF: {item['host']} waits "
        f"{backoff} slot(s) before retransmission\n"
    )
```

```
time += 1
```

```
log.write("\n" + "=" * 50 + "\n")
log.write("SIMULATION SUMMARY\n")
log.write(f'Total frames generated : {TOTAL_FRAMES}\n')
log.write(f'Successful transmissions: {successful}\n')
log.write(f'Collision events      : {collisions}\n')
log.write(f'Simulation time slots  : {time}\n')
```

```
print("CSMA/CD simulation completed successfully.")
print("Log file created: csma_log.txt")
print("\nSimulation Summary")
print("-----")
print("Total frames generated :", TOTAL_FRAMES)
print("Successful transmissions:", successful)
print("Collision events      :", collisions)
print("Simulation time slots  :", time)
```


OUTPUT:

Run

csma_cd_simulation

csma_cd_simulation.py

PS C:\Users\HP\Desktop\CSMA_CD> python csma_cd_simulation.py
CSMA/CD simulation completed successfully.
Log file created: csma_log.txt

Simulation Summary

Total frames generated : 12
Successful transmissions: 12
Collision events : 5
Simulation time slots : 18

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

csma_log.txt - Notepad

File Edit Format View Help
CSMA/CD SIMULATION LOG
=====

Time Slot 0
TRANSMISSION ATTEMPT: Host 1 Frame 1 Attempt 1
TRANSMISSION ATTEMPT: Host 3 Frame 3 Attempt 1
COLLISION: 2 hosts attempted transmission simultaneously
BACK-OFF: Host 1 waits 1 slot(s) before retransmission
BACK-OFF: Host 3 waits 0 slot(s) before retransmission

Time Slot 1
TRANSMISSION ATTEMPT: Host 3 Frame 3 Attempt 2
SUCCESS: Host 3 transmitted Frame 3

Time Slot 2
TRANSMISSION ATTEMPT: Host 1 Frame 1 Attempt 2
SUCCESS: Host 1 transmitted Frame 1

Time Slot 3
TRANSMISSION ATTEMPT: Host 2 Frame 2 Attempt 1
SUCCESS: Host 2 transmitted Frame 2

Time Slot 4
TRANSMISSION ATTEMPT: Host 4 Frame 4 Attempt 1
TRANSMISSION ATTEMPT: Host 1 Frame 5 Attempt 1
COLLISION: 2 hosts attempted transmission simultaneously
BACK-OFF: Host 4 waits 0 slot(s) before retransmission
BACK-OFF: Host 1 waits 1 slot(s) before retransmission

Time Slot 5
TRANSMISSION ATTEMPT: Host 4 Frame 4 Attempt 2
SUCCESS: Host 4 transmitted Frame 4

Time Slot 6
TRANSMISSION ATTEMPT: Host 1 Frame 5 Attempt 2
SUCCESS: Host 1 transmitted Frame 5
...

=====

SIMULATION SUMMARY
Total frames generated : 12
Successful transmissions: 12
Collision events : 5
Simulation time slots : 18

Ln 1, Col 1 100% Windows (CRLF) UTF-8

DIAGRAM:

CSMA/CD PROTOCOL SIMULATION IN STAR TOPOLOGY

1. STAR TOPOLOGY DIAGRAM

```
graph TD; H1[Host 1] --- S[Switch]; H2[Host 2] --- S; H3[Host 3] --- S; H4[Host 4] --- S;
```

Star Topology Explanation
In this topology, four hosts are connected to a central switch. All communication between hosts passes through the switch. The switch acts as a central point that forwards frames to the destination host.

2. PROGRAM EXECUTION OUTPUT

```
C:\Windows\System32\cmd.exe - python csma_cd_simulation.py -  
C:\>python csma_simulation.py  
CSMA/CD simulation completed successfully.  
Log file created: csma_log.txt  
  
Simulation Summary  
-----  
Total frames generated : 12  
Successful transmissions: 12  
Collision events : 5  
Simulation time slots : 18  
  
Press any key to continue . . . _
```

3. LOG FILE (csma_log.txt)

csma_log.txt - Notepad

File Edit Format View Help
CSMA/CD SIMULATION LOG
=====

Time Slot 0
TRANSMISSION ATTEMPT: Host 1 Frame 1 Attempt 1
TRANSMISSION ATTEMPT: Host 3 Frame 3 Attempt 1
COLLISION: 2 hosts attempted transmission simultaneously
BACK-OFF: Host 1 waits 1 slot(s) before retransmission
BACK-OFF: Host 3 waits 0 slot(s) before retransmission

Time Slot 1
TRANSMISSION ATTEMPT: Host 3 Frame 3 Attempt 2
SUCCESS: Host 3 transmitted Frame 3

Time Slot 2
TRANSMISSION ATTEMPT: Host 1 Frame 1 Attempt 2
SUCCESS: Host 1 transmitted Frame 1

Time Slot 3
TRANSMISSION ATTEMPT: Host 2 Frame 2 Attempt 1
SUCCESS: Host 2 transmitted Frame 2

Time Slot 4
TRANSMISSION ATTEMPT: Host 4 Frame 4 Attempt 1
TRANSMISSION ATTEMPT: Host 1 Frame 5 Attempt 1
COLLISION: 2 hosts attempted transmission simultaneously
BACK-OFF: Host 4 waits 0 slot(s) before retransmission
BACK-OFF: Host 1 waits 1 slot(s) before retransmission

Time Slot 5
TRANSMISSION ATTEMPT: Host 4 Frame 4 Attempt 2
SUCCESS: Host 4 transmitted Frame 4

Time Slot 6
TRANSMISSION ATTEMPT: Host 1 Frame 5 Attempt 2
SUCCESS: Host 1 transmitted Frame 5
...

=====

SIMULATION SUMMARY
Total frames generated : 12
Successful transmissions: 12
Collision events : 5
Simulation time slots : 18

Ln 1, Col 1 100% Windows (CRLF) UTF-8

4. WORKING OF CSMA/CD (STEP BY STEP)

1. A host that has a frame to send first senses the channel.
2. If the channel is idle, the host starts transmitting.
3. If the channel is busy, the host waits until it becomes idle.
4. If two or more hosts start transmitting at the same time, a collision occurs.
5. All hosts involved in the collision detect it and stop transmitting immediately.
6. Each host calculates a random back-off time using binary exponential back-off:
 - Back-off range = 0 to $(2^k - 1)$, where k = number of collisions.
7. After waiting for the back-off time, the host again senses the channel.
8. If the channel is idle, the host retransmits the frame.
9. This process continues until the frame is transmitted successfully.

5. HOW CSMA/CD MINIMIZES CONFLICTS

- **Carrier Sense:** Hosts listen to the channel before transmitting to avoid unnecessary collisions.
- **Collision Detection:** If a collision occurs, it is detected quickly and the transmission is stopped.
- **Random Back-off:** Instead of retransmitting immediately, hosts wait for a random time. This reduces the chance that the same hosts collide again.
- **Binary Exponential Back-off:** The waiting range increases with each collision, which further reduces repeated collisions in a congested network.
- **Efficient Channel Utilization:** The channel is used efficiently because frames are transmitted only when the channel is free, and collisions are minimized.

6. CONCLUSION

The simulation successfully demonstrates the operation of CSMA/CD protocol in a star topology with four hosts. The program detects collisions, calculates random back-off intervals, and retransmits frames after waiting. The log file records all events including transmission attempts, collisions, back-offs, and successful transmissions. CSMA/CD minimizes data transmission conflicts by combining carrier sensing, collision detection, and random back-off, ensuring fair access to the shared medium and improving network performance.

Figure: CSMA/CD Protocol Simulation with Four Hosts in a Star Topology

EXPLANATION:

- The diagram represents the simulation of the **CSMA/CD protocol in a star topology** consisting of four hosts connected to a central switch.
- Each host can attempt to transmit a frame through the network.
- Before transmission, the host senses the channel to determine whether it is available.
- If two or more hosts attempt transmission simultaneously, a **collision** occurs.
- The collision is detected, and the affected hosts stop transmission and calculate a **random back-off interval** using the binary exponential back-off method.
- After waiting for the calculated interval, the hosts attempt retransmission.
- The program records every transmission attempt, collision, back-off interval, and successful transmission in the **csma_log.txt** file.
- This process reduces repeated collisions and improves efficient access to the shared communication medium.

4) Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

ANSWER:

Program:

```
import socket
import threading

mac_table = {}
hosts = {}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 5000))
server.listen(2)

print("Switch started...")
print("Waiting for hosts...\n")

def handle_client(conn, port):

    mac = conn.recv(1024).decode()

    mac_table[mac] = port
    hosts[mac] = conn

    print("Host connected")
    print("MAC Address:", mac)
    print("Port:", port)

while True:
    try:
        data = conn.recv(1024).decode()
```

```

if not data:
    break

destination_mac, message = data.split("|", 1)

print("\nFrame received")
print("Source MAC:", mac)
print("Destination MAC:", destination_mac)
print("Message:", message)

if destination_mac in hosts:
    destination = hosts[destination_mac]

    destination.send(
        ("Message from " + mac + ": " + message).encode()
    )

    print("Frame forwarded successfully.")

else:
    print("Destination MAC not found.")

print("\nMAC Address Table")
print("-----")

for address, p in mac_table.items():
    print(address, "-> Port", p)

print("-----")

except:
    break

```

```
conn.close()
```

```
while len(hosts) < 2:  
    conn, address = server.accept()
```

```
    port = len(hosts) + 1
```

```
    thread = threading.Thread(  
        target=handle_client,  
        args=(conn, port)  
    )
```

```
    thread.start()
```

2)HOST/CLIENT:

```
import socket  
import threading
```

```
server = ("localhost", 5000)
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
sock.connect(server)
```

```
mac = input("Enter MAC Address: ")  
sock.send(mac.encode())
```

```
print("Connected to switch.")
```

```
print("Message format: destination_mac|message")
print("Example: BB:BB:BB:BB:BB:BB|Hello\n")
```

```
def receive():
    while True:
        try:
            data = sock.recv(1024).decode()

            if data:
                print("\n" + data)

        except:
            break
```

```
threading.Thread(target=receive, daemon=True).start()
```

```
while True:

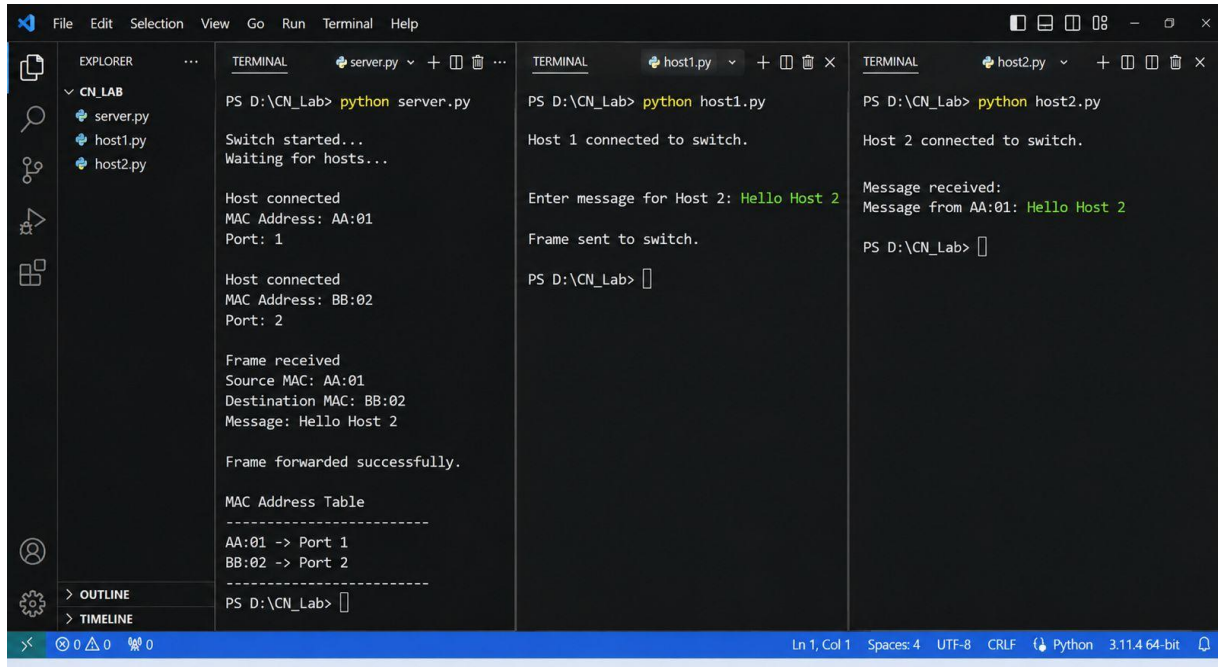
    message = input(">> ")

    if message.lower() == "exit":
        break

    sock.send(message.encode())

sock.close()
```

OUTPUT:



```
File Edit Selection View Go Run Terminal Help

EXPLORER
CN_LAB
  server.py
  host1.py
  host2.py

TERMINAL
server.py
PS D:\CN_Lab> python server.py

Switch started...
Waiting for hosts...

Host connected
MAC Address: AA:01
Port: 1

Host connected
MAC Address: BB:02
Port: 2

Frame received
Source MAC: AA:01
Destination MAC: BB:02
Message: Hello Host 2

Frame forwarded successfully.

MAC Address Table
-----
AA:01 -> Port 1
BB:02 -> Port 2
-----
PS D:\CN_Lab>

TERMINAL
host1.py
PS D:\CN_Lab> python host1.py

Host 1 connected to switch.

Enter message for Host 2: Hello Host 2

Frame sent to switch.

PS D:\CN_Lab>

TERMINAL
host2.py
PS D:\CN_Lab> python host2.py

Host 2 connected to switch.

Message received:
Message from AA:01: Hello Host 2

PS D:\CN_Lab>
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python 3.11.4 64-bit

DIAGRAM:

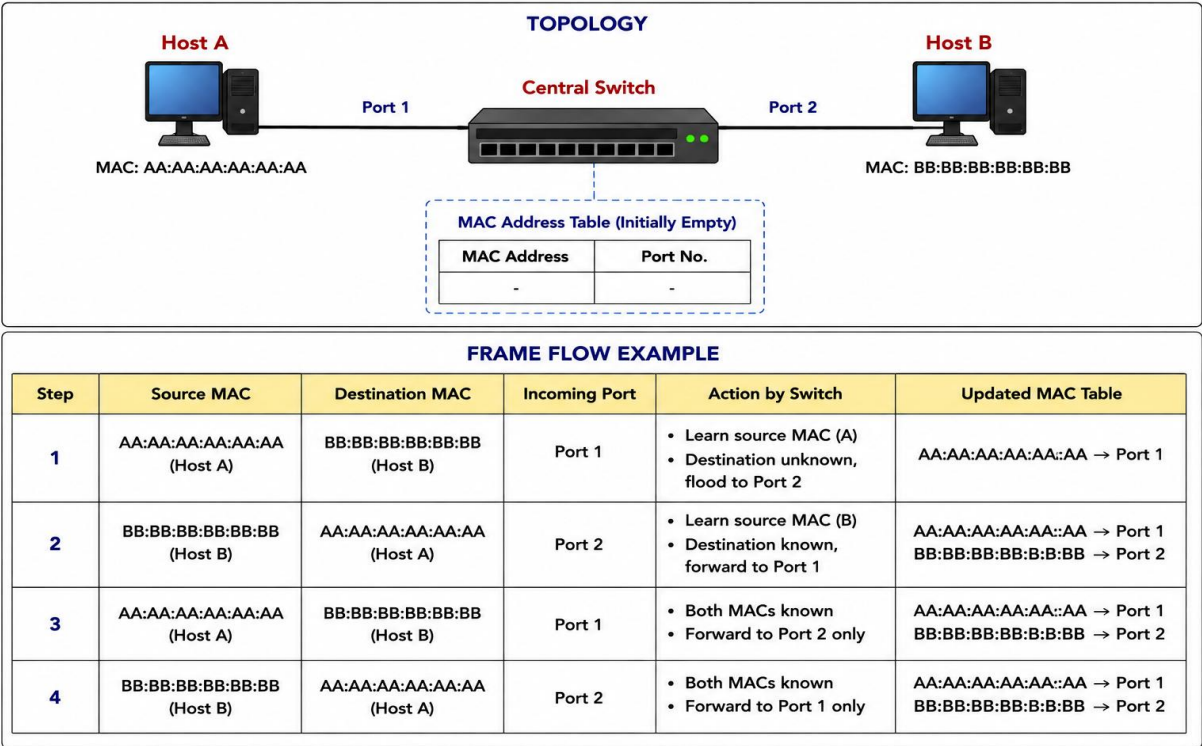


Figure: Star Topology and Frame Flow in a Client–Server Ethernet Network

EXPLANATION:

- The figure illustrates two hosts, Host A and Host B, connected to a central switch through Port 1 and Port 2, respectively.
- Host A and Host B have unique MAC addresses. The lower section shows the frame flow between the two hosts.
- When Host A sends a frame to Host B, the frame enters the switch through Port 1 and is forwarded toward Host B through Port 2. Similarly, when Host B sends a frame to Host A, the frame enters through Port 2 and is forwarded through Port 1.
- The figure also shows how the switch updates its MAC Address Table and uses the learned port information to forward subsequent frames to the correct destination.

